Roteiro do vídeo – Cinto Inteligente (alvo de 14 minutos e 45 segundos)

O tempo calculado é de cerca de 140 palavras por minuto, o que dá cerca de 14 min 45 s, perto do limite de 15 minutos. Falem num ritmo firme e cortem pausas longas na edição. Todos falam ao menos uma vez e se apresentam com nome completo e função na primeira fala.

Tempo	Bloco do roteiro do curso	Quem fala
00:00–01:15	Identificação, equipe, vertente	Kayky
01:15–02:30	Problema e hipótese	Aluisio
02:30–04:00	Arquitetura IoT e protocolos	Erick
04:00–06:30	Hardware na bancada	Enzo
06:30–08:00	Dataset, pipeline e atributos da IA	Aluisio
08:00–10:30	Modelo, validação, métricas e inferência	Erick
10:30–13:30	Demonstração de ponta a ponta	Kayky, Enzo e Erick
13:30–14:45	Resultados, limitações e encerramento	Kayky

BLOCO 1 – 00:00 A 01:15 · KAYKY

"Olá! Eu sou **Kayky Crespo dos Santos**, Product Owner, gerente de projeto e Scrum Master da equipe. Este é o projeto **Cinto Inteligente com IoT e Inteligência Artificial para Monitoramento de Atividades, Sinais Vitais e Localização Indoor**, um sistema de detecção de possíveis quedas. A equipe é formada por Aluisio Pereira Alves, Enzo Orlandi Gomes, Erick de Brito Carvalho e por mim, com orientação do professor Marco Gomes. O projeto está na vertente **SmartCity, Cidades Inteligentes e Infraestrutura Urbana**. As cidades estão envelhecendo, e o cuidado com idosos virou um **serviço urbano**: casas de repouso, unidades de saúde e atendimento domiciliar dependem de equipes que não conseguem vigiar todos o tempo todo. Nossa proposta é uma **infraestrutura de sensoriamento** para esses espaços: antenas fixas no prédio formam uma pequena rede de sensores e transmitem por **MQTT**, o protocolo de telemetria usado em cidades inteligentes. O processamento é **local, sem nuvem**, e o alerta chega à equipe de cuidado em segundos, com a região do ambiente onde a pessoa caiu. A mesma rede pode ser replicada em vários prédios e, no futuro, ligada a centrais de teleassistência do município. Agora o Aluisio apresenta o problema e a nossa hipótese."

Na tela: capa com título, logo da UNIP e a palavra **“Vertente: SmartCity”** em destaque. Cada integrante aparece na câmera com uma tarja de nome e função ao falar pela primeira vez. Ao falar da vertente, um slide **“Por que SmartCity”** com três ícones:

1. **Serviço urbano:** casa de repouso, unidade de saúde e casa.
2. **Infraestrutura de sensores:** antenas no prédio transmitindo por MQTT.
3. **Escala:** vários prédios ligados a uma central de cuidado.

BLOCO 2 – 01:15 A 02:30 · ALUISIO

“Eu sou **Aluisio Pereira Alves**, pesquisador e redator do projeto. O problema que atacamos é o **tempo entre uma queda e o momento em que alguém percebe que ela aconteceu**. Hoje, muitas soluções dependem de a própria pessoa apertar um botão de emergência, o que nem sempre é possível depois de cair. Outras usam GPS, que falha dentro de casa, ou disparam alarmes falsos a cada movimento brusco. No nosso sistema, o **alerta é automático**, não depende de quem caiu. **Não usamos câmeras**, o que preserva a privacidade. E aplicamos **inovação frugal**: componentes de baixo custo e fáceis de encontrar, como o ESP32 e sensores comuns, para que a solução seja acessível a casas de repouso e famílias. Nossa **hipótese** é que um cinto com sensor inercial e um modelo Random Forest detecta quedas com alta sensibilidade e poucos alarmes falsos. Escolhemos o Random Forest porque, com o SisFall, ele teve **alta sensibilidade**, **resistiu ao sobreajuste** ao ser testado em voluntários que não estavam no treino, **lidou bem com a variação entre pessoas**, **mostrou a importância de cada atributo** e funcionou com **pouco ajuste fino**: definimos apenas três parâmetros. Treinado e validado com o **dataset SisFall**, com 38 voluntários, o modelo detectou **98,8% de 1.798 quedas** e acertou **95,3% de 2.707 atividades normais**. Erick, a arquitetura.”

Na tela: um slide com duas colunas.

- **Esquerda, “Como é hoje”:** botão de emergência (depende de quem caiu), GPS (falha em ambiente interno) e alarmes falsos.

- **Direita, “Nossa proposta”:** alerta automático, sem câmeras e inovação frugal.

Em seguida, a hipótese com os números em destaque: **98,8% de 1.798 quedas** e **95,3% de 2.707 atividades**.

BLOCO 3 – 02:30 A 04:00 · ERICK

“Eu sou **Erick de Brito Carvalho**, arquiteto de soluções e desenvolvedor de software. O ecossistema tem cinco camadas. Na **borda** está o cinto: um ESP32-C3 lê o MPU6050 pelo barramento **I²C** a 400 kHz e o módulo de ECG AD8232 por uma entrada **analógica** e dois pinos **GPIO** que indicam eletrodo solto. São cem leituras por segundo. Na **conectividade**, o cinto usa Bluetooth Low Energy de duas formas ao mesmo tempo. Uma **conexão GATT** envia os sensores a cem hertz para uma das antenas. Um **anúncio** a cada cem milissegundos é escutado por todas as antenas, que medem a intensidade do sinal, o RSSI. As três antenas são ESP32 DevKit e repassam tudo por **Wi-Fi, sobre TCP/IP, usando MQTT**, até o broker Mosquitto no computador central. Cada antena também publica se está online ou offline, pela mensagem de *last will* do MQTT. No computador, a aplicação Python recebe o MQTT e grava no **SQLite**, que é a persistência. Ela também roda o modelo e a localização, que são a inteligência, e mostra tudo num **painel Streamlit**, com alertas pelo **Telegram**, que são a interface. Agora o Enzo mostra o hardware.”

Na tela:

- `docs/esquemas/arquitetura_camadas.png`, em tela cheia. Use zoom ou um destaque em cada camada enquanto ela é citada.
- No fim, por 5 segundos, o fluxograma `docs/IMAGENS/FLUXOGRAMA.jpeg`.

BLOCO 4 – 04:00 A 06:30 · ENZO (BANCADA)

“Eu sou **Enzo Orlandi Gomes**, responsável pelo sistema embarcado. Este é o cinto aberto. Aqui está o **ESP32-C3 Super Mini**, o cérebro do cinto. Este é o **MPU6050**, acelerômetro e giroscópio de três eixos, ligado no 3,3 volts, com o SDA no GPIO 8 e o SCL no GPIO 9. Configuramos o sensor para mais ou menos 8 g e 2000 graus por segundo, as mesmas faixas

usadas no treino do modelo. Este é o **AD8232**, do ECG. A saída analógica entra no GPIO 4, e os pinos LO menos e LO mais, nos GPIOs 2 e 3, avisam quando um eletrodo solta. Os três eletrodos chegam por este conector P2, então dá para trocar o cabo sem abrir a caixa. Na **alimentação**, temos duas baterias 18650 de 1000 mAh em paralelo e este **TP4056**, que carrega pela USB-C e protege as baterias. Depois vêm esta **chave liga-desliga** e o **MT3608**, que eleva os 3,7 volts da bateria para 5 volts estáveis para o ESP32. Os sensores recebem 3,3 volts do regulador da própria placa. No pior caso, o cinto consome cerca de 355 mA e dura em torno de 3 horas. A antena dura cerca de uma hora e meia, por isso pode ficar ligada no carregador. Tudo fica nesta **caixa impressa em 3D**, com roscas de latão M3 para abrir e fechar sem espanar. Ela vai presa na frente do corpo, na altura da cintura, a mesma posição do dataset. Esta é a **antena**: a mesma alimentação, com uma bateria, e um ESP32 DevKit V1. Ela não tem sensores; o trabalho dela é só rádio. O firmware é gravado pela Arduino IDE, com o pacote ESP32 da Espressif. A antena precisa do driver CP210x; o C3 tem USB nativo. No topo do código configuramos o código do ambiente, o número da antena, a rede e o IP do broker. No monitor serial, a antena mostra o Wi-Fi e o MQTT conectados e, aqui, **'conectado e autenticado'** com o cinto, recebendo cem pacotes por segundo. Aluisio, os dados da IA."

Na tela:

1. **Câmera de cima** no cinto aberto, com o dedo apontando cada módulo. Se possível, ponha uma etiqueta numerada em cada um, igual à imagem do case.
2. Ao falar dos pinos, mostre por 3 a 4 segundos o esquema elétrico do cinto (Seção 2.3 do manual), com zoom nas ligações do MPU e do AD8232.
3. Ao falar da alimentação, mostre o multímetro medindo **5,0 V** na saída do MT3608. Isso comprova o barramento de alimentação, que o roteiro do curso pede.
4. O cinto fechado sendo preso ao cinto da pessoa e a antena na parede ou no suporte.
5. Na Arduino IDE, os `#define` do cinto: pinos e `FREQ_LEITURA_HZ`.

6. O Monitor Serial da antena com a linha `[GATT] CONECTADO E AUTENTICADO!` e a linha `[STATUS]`.

Atenção: o código da antena tem a senha real do Wi-Fi (linhas 40–41) e o `alerta_telegram.py` tem o token (linha 23). **Não mostre essas linhas**: role a tela ou borre na edição.

BLOCO 5 – 06:30 A 08:00 · ALUISIO

"O objetivo da IA é **classificar cada janela de movimento como possível queda ou atividade normal**. Todo o pipeline foi feito em **Python com scikit-learn**; este diagrama mostra as duas partes: o treino, feito uma vez, e a operação em tempo real. Para treinar, usamos o **SisFall**, um dataset público publicado por Sucerquia e colegas em 2017. Ele tem **38 voluntários**, 23 adultos jovens e 15 idosos, com **1.798 quedas simuladas** e **2.707 atividades do dia a dia**, como andar, sentar, deitar e subir escada, gravadas com o sensor na cintura a 200 hertz. Como o nosso sensor é diferente, o script de treino adapta os dados ao cinto. Ele reduz de 200 para 100 hertz e limita os valores a mais ou menos 8 g e 2000 graus por segundo, as mesmas faixas do MPU6050. Depois dividimos os sinais em **janelas de 2,5 segundos**, com passo de meio segundo, e chegamos a **122.572 janelas**. De cada janela extraímos **8 atributos**:

- aceleração máxima e mínima;
- desvio da aceleração;
- rotação máxima e desvio da rotação;
- inclinação entre o início e o fim da janela;
- *jerk*, a variação brusca da aceleração;
- o quanto a pessoa fica parada no final.

Como usamos a magnitude, os atributos não dependem de como o sensor está virado. E **a mesma função calcula os atributos no treino e em tempo real**. Erick, o modelo."

Na tela:

- **Abertura do bloco:** `docs/esquemas/pipeline_ia.png` em tela cheia. Destaque cada caixa da faixa “Treino” enquanto ela é citada.
- Slide com os números do SisFall e um gráfico de sinal de uma queda: um pico de aceleração seguido de repouso. Dá para capturar da aba Detalhes, em “Gráficos de Sinais”, durante um teste.
- Em `features.py`, a lista `FEATURES` com os 8 atributos (linhas 32–41).
- Em `scripts/treino_sisfall.py`, a função `load_trial()`, com a reamostragem e a saturação.

BLOCO 6 – 08:00 A 10:30 · ERICK

“Como o Aluisio adiantou, usamos o **Random Forest** do scikit-learn, um conjunto de árvores de decisão que votam. Comparado a uma regra de limiar, ele erra menos em movimentos bruscos. Comparado a redes neurais, precisa de menos dados e roda em milissegundos num computador comum. No código de treino usamos **200 árvores**, no mínimo **2 amostras por folha** para evitar sobreajuste e **pesos balanceados**, porque só 23% das janelas são de queda. A validação é cruzada em **5 partes, separando por voluntário, com GroupKFold**: quem está no teste nunca aparece no treino. Isso evita vazamento entre janelas sobrepostas e simula uma pessoa que o modelo nunca viu. Na **matriz de confusão**, de 1.798 quedas o modelo detectou **1.777** e perdeu 21. De 2.707 atividades, gerou **128 alarmes falsos**. Isso dá:

- **recall** de 98,8%;
- especificidade de 95,3%;
- **precisão** de 93,3%;
- **F1** de 96%;
- acurácia de 96,7%.

A inclinação foi o atributo mais importante. O limiar padrão é 0,5, mas pode ser ajustado no painel: em 0,7, os alarmes falsos caem e a sensibilidade cai um pouco. O modelo treinado é salvo com **joblib** e carregado pelo painel. Em **tempo real**, o detector junta as amostras de cada cinto e, a cada meio segundo, monta a janela, calcula os atributos e pede a probabilidade ao

modelo. Se ela passa do limiar, a queda é registrada, e por 5 segundos o mesmo cinto não gera outro alarme, para não duplicar alertas. Aqui está a inferência em **dados reais do nosso cinto**: este script reproduz uma gravação feita com o MPU6050 como se ela chegasse ao vivo, e o modelo aponta o momento da queda, com a probabilidade, a aceleração máxima e a inclinação. Vamos ver tudo funcionando."

Na tela:

1. Em `scripts/treino_sisfall.py`, a função `make_model()` (linhas 98–100) e o laço `GroupKFold` (linha 136).
2. A **matriz de confusão**: `docs/esquemas/matriz_confusao.png`.
3. O painel de métricas, com limiar e importância dos atributos: `docs/esquemas/metricas_modelo.png`. Mostre em seguida `modelos/resultado_treino_mvp_8g.txt` por alguns segundos, para provar a origem dos números.
4. Em `detector_queda.py`, o método `adicionar()` da classe de tempo real (linha 155). Volte por 3 segundos ao `pipeline_ia.png`, destacando a faixa "Operação em tempo real".
5. **Obrigatório (prova de inferência em dados do sensor):** no terminal, rode `python scripts/reproduzir_csv.py dados/gravacoes/<arquivo>.csv --modelo modelos/modelo_mvp_8g.joblib`. Use uma gravação feita antes com "Gravar amostras" ligado e uma queda simulada no colchão. O script mostra o total de janelas analisadas e, para cada queda detectada, o horário, a probabilidade, a aceleração máxima, a inclinação e a rotação máxima.

BLOCO 7 – 10:30 A 13:30 · DEMONSTRAÇÃO DE PONTA A PONTA

Kayky:

"Agora o caminho completo: sensor, borda, transmissão, banco e painel. Este é o painel, com o broker conectado e as três antenas online."

Enzo:

"Vou ligar o cinto. Uma antena já conectou por GATT. Na aba Detalhes aparece a antena em GATT e as outras em anúncio, cada uma com o seu RSSI, e a taxa está perto de cem amostras por segundo. Agora os

eletrodos: o status mudou para conectado e o LED apagou. Este é o sinal de ECG.”

Erick:

“Os dados passam pelo broker. Cada linha é um pacote MQTT com o RSSI e as leituras do sensor. No banco SQLite, a sessão foi aberta e cada janela classificada é gravada com os atributos e a probabilidade.”

Kayky:

“Na aba Monitoramento, o gráfico mostra a probabilidade de queda das últimas janelas. Andando e sentando, ela fica baixa. Agora o Enzo vai simular uma queda no colchão.”

(o Enzo simula a queda)

Kayky:

“O cartão do cinto mudou para **possível queda**, com a probabilidade, data, hora, a antena mais próxima e o **quadrante** do ambiente. A queda entrou na tabela de quedas detectadas e o alerta chegou no Telegram.”

Erick:

“No banco, a queda ficou registrada junto com a janela que a gerou e com o status do envio ao Telegram. No mapa aparecem o raio estimado de cada antena e a posição do cinto. Como o corpo atenua o sinal Bluetooth, tratamos a localização como uma região do ambiente, um quadrante, e não como um ponto exato. Tudo pode ser exportado em CSV.”

Na tela (tela dividida é o ideal):

- **Esquerda:** câmera na pessoa com o cinto, mostrando o colchão e o ambiente com as antenas.
- **Direita:** o painel Streamlit (http://localhost:8501).
- **Detalhe no canto:** o celular com o Telegram, filmado ou espelhado.

Comandos e consultas prontos:

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" -h localhost -t "oqpnb/#" -v
```

No **DB Browser for SQLite**, abra `dados/oqpnb.db` em modo somente leitura e rode:

```
SELECT instante, amax, tilt, probabilidade, alarme FROM janela ORDER BY id_janela DESC LIMI
T 10;

SELECT q.id_queda, j.instante, j.probabilidade, q.id_antena, q.quadrante, q.telegram_envio
FROM queda q JOIN janela j ON j.id_janela = q.id_janela
ORDER BY q.id_queda DESC LIMIT 5;
```

Os horários no banco estão em UTC, ou seja, 3 horas a mais que o horário de Brasília. Vale comentar isso se aparecer na tela.

Sequência sugerida na tela:

1. Aba Detalhes: Status do Sistema, cartões das antenas (GATT ou anúncio) e tabela RSSI.
2. Status do eletrodo e gráfico do ECG.
3. Terminal com o `mosquitto_sub` rodando.
4. DB Browser com as janelas.
5. Aba Monitoramento: gráfico de probabilidade enquanto a pessoa anda.
6. Queda, cartão vermelho e tabela de quedas.
7. Notificação no Telegram.
8. DB Browser com a queda registrada.
9. Mapa com os círculos e o marcador do cinto.
10. Botão de exportação.

BLOCO 8 – 13:30 A 14:45 · KAYKY

"Para fechar: mostramos, funcionando, o caminho completo, do sensor na cintura ao alerta no celular, passando por Bluetooth, Wi-Fi, MQTT, banco de dados, inteligência artificial e painel web. O **nosso Random Forest**, treinado com o SisFall e testado em voluntários que ele nunca tinha visto, detectou **98,8% das quedas**, com **F1 de 96%**. Na prática, o cinto funcionou em tempo real, sem câmeras e com componentes de baixo custo. Sabemos onde ainda precisamos evoluir. O próximo passo é validar o modelo com quedas gravadas pelo nosso próprio cinto. Também queremos melhorar a localização: o corpo atenua o sinal, e por isso hoje indicamos a região do ambiente, não o ponto exato. E precisamos aumentar a autonomia da bateria. O ECG segue experimental, sem finalidade médica. Nosso objetivo é que **nenhuma queda passe despercebida**, em casa, numa casa de

repouso ou em qualquer prédio da cidade. O passo a passo completo está no nosso Guia do Usuário. Muito obrigado!”

Na tela: um slide em três colunas.

- **“Entregamos”:** cinto, 3 antenas, painel e alerta no Telegram.
- **“Resultados”:** 98,8% das quedas e F1 de 96%, no SisFall com o nosso modelo.
- **“Próximos passos”:** dados próprios, localização e bateria.

Na frase final, a equipe aparece junta na câmera, com o cinto e uma antena na mão. Por último, a capa do manual em PDF e os nomes dos quatro integrantes.

DICAS DE PRODUÇÃO

- **Grave por blocos e edite depois.** É mais fácil repetir um trecho do que o vídeo inteiro. Nos blocos 1 e 2, quem fala lê o texto com o teleprompter no celular.
- **Ensaiem a demonstração duas vezes antes.** Carreguem as baterias, deixem as antenas no carregador e liguem o painel uns 5 minutos antes de gravar.
- **Plano B se o hardware falhar:** grave antes uma sessão com “Gravar amostras” ligado, incluindo uma queda. Se algo der errado no dia, use o `reproduzir_csv.py` para mostrar o modelo funcionando com os dados reais do sensor.
- **Queda com segurança:** use colchão ou sofá e um movimento controlado (sentar rápido e deitar de lado). Ninguém deve cair de verdade.
- **Antes de gravar:**
  - Teste o envio pelo botão “Testar envio”.
  - Feche notificações do computador.
  - Use zoom de 125 a 150% no navegador e no editor de código, para o texto ficar legível no vídeo.
- **Hospedagem:** YouTube “não listado” ou Google Drive com acesso por link, como o roteiro do curso pede.

IMAGENS DE APOIO PARA OS SLIDES

Ficam em `docs/esquemas/` (PNG para slides e SVG para ampliar sem perder qualidade), exceto o fluxograma, que fica em `docs/IMAGENS/`:

Bloco	Arquivo	Conteúdo
3	`arquitetura_camadas.png`	Arquitetura em 5 camadas
3	`IMAGENS/FLUXOGRAMA.jpeg`	Fluxo de um ciclo de leitura até o alerta
4	Esquemas elétricos do cinto e da antena (em elaboração)	Ligações dos módulos
5 e 6	`pipeline_ia.png`	Pipeline de IA: treino offline e operação em tempo real
6	`matriz_confusao.png`	Matriz de confusão (VP, FN, FP, VN)
6	`metricas_modelo.png`	Recall, precisão, F1, especificidade, acurácia, efeito do limiar e importância dos atributos